

KAPITEL 8: AJAX

LERNZIELE

- Das AJAX-Prinzip erklären können.
- Den Unterschied zwischen klassischem Ansatz und AJAX erklären können.
- Die XMLHttpRequest-API in wesentlichen Bestandteilen kennen und anwenden können.
- Die wesentlichen Vor- und Nachteile der AJAX-Anwendung kennen.



8.1 AJAX

Das Standard-Modell für Webanwendungen sieht vor, dass Webseiten von einem Webserver an den Browser gesendet und dort angezeigt werden.

Jede wesentliche Interaktion mit der Webseite (Formular-Submit, Navigation) führt zum erneuten Aufruf des Webservers, der dann eine neue Seite liefert.

Für interaktive Anwendungen ist dieses Anfrage/Antwort-Schema **nicht geeignet**, da jeder Seitenwechsel mit einer Verzögerung einhergeht, die die Benutzererfahrung beeinträchtigt.

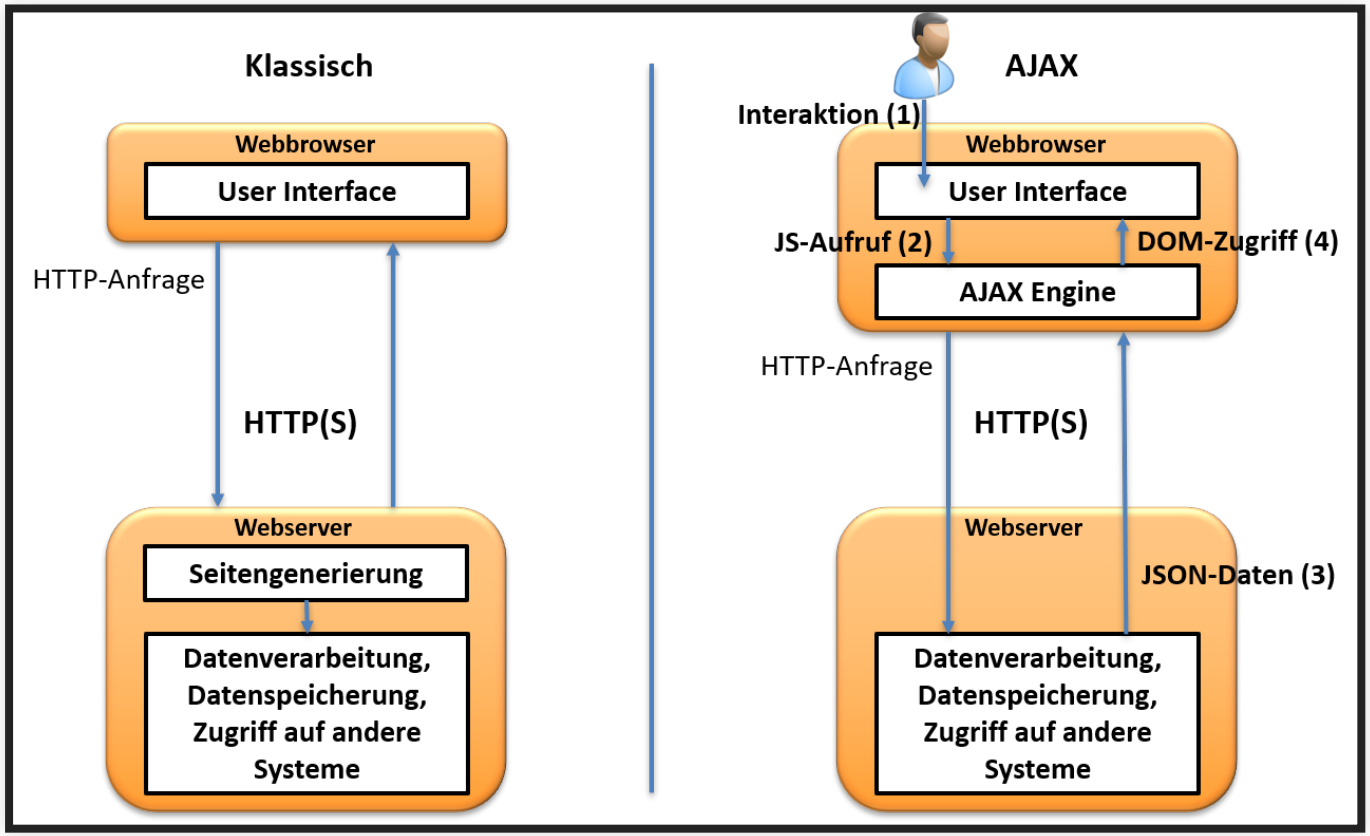
AJAX-Ansatz zur Lösung des Problems

Asynchronous JavaScript and XML

1. Der Browser sendet einen asynchronen Aufruf an einen HTTP-Server
2. Der Server antwortet mit einer XML- oder JSON-Nachricht.
3. Der Inhalt der Nachricht wird in das DOM geschrieben damit sofort angezeigt.

Im Ergebnis unterbleibt das unerwünschte Seitennachladen, Daten werden scheinbar sofort sichtbar bzw. auf dem Server gespeichert.

Ein Vergleich der klassischen mit der AJAX-Vorgehensweise ist nachfolgend dargestellt.

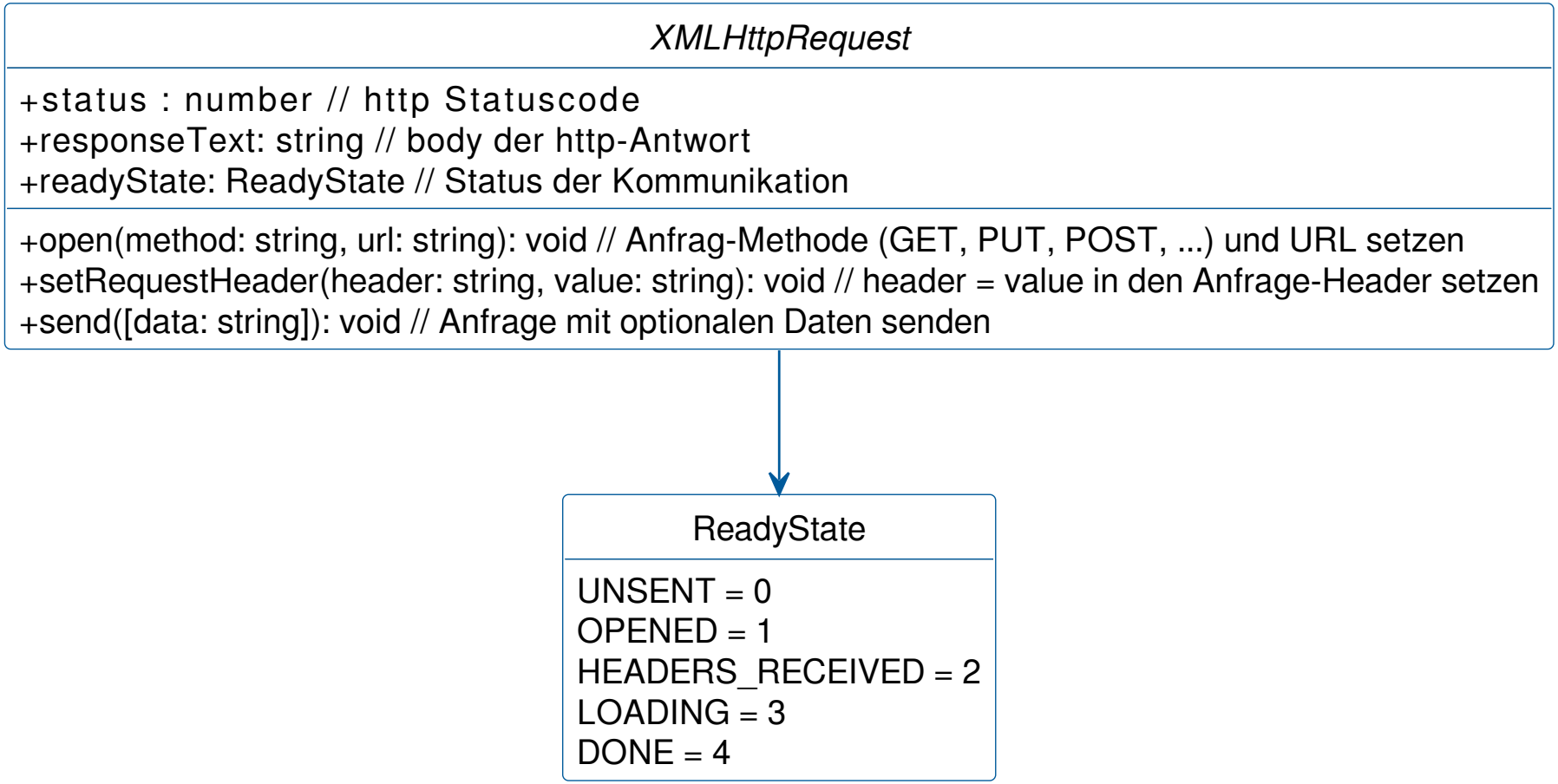


DAS XMLHTTPREQUEST-OBJEKT

Technisch möglich gemacht wurde AJAX durch die Einführung des XMLHttpRequest-Objekts, welches durch die Browser interpretiert und von JavaScript über eine API genutzt werden kann.

Die API und weitere Informationen dazu findet man z.B. bei selfhtml.

Nachfolgend ein komprimiertes Modell mit den wesentlichen Eigenschaften:





Die API wird folgendermaßen genutzt (Beispiel):

```
let xhr = new XMLHttpRequest();
xhr.onreadystatechange = function () {
    if (xhr.readyState == 4) {
        if (xhr.status == 200) {
            document.getElementById("txtHint").innerHTML = xhr.responseText;
        } else {
            alert('Fehler beim Laden: status = ' + xhr.status);
        }
    }
};
xhr.open("GET", "gethint.php?q=" + str);
xhr.send();
```

- Erzeugen des Objekts und Installieren eines Handlers für den Fall, dass sich der readyState nach dem Senden der Nachricht ändert.
- Aufsetzen der Nachricht, Methode "GET" an "gethint.php", Query String: q=<string>
- Senden der Nachricht
- Bei Änderung des readyState wird geprüft, ob eine gültige (HTTP-Code 200) Antwort eingetroffen ist (readyState 4).
- Verarbeiten des Nachrichtenbodies (responseText)

Alternative zu `onreadystatechange`: `onload`

In der Regel interessiert man sich lediglich für `readyState 4`, daher gibt es eine kürzere Variante durch Verwendung von `onload`:

```
let xhr = new XMLHttpRequest();
xhr.onload = function () {
    // hier gilt: readyState == 4 (=Nachricht/Fehler vollständig empfangen)
    if (xhr.status == 200) {
        document.getElementById("txtHint").innerHTML = xhr.responseText;
    } else {
        alert('Fehler beim Laden: status = ' + xhr.status);
    }
};
xhr.open("GET", "gethint.php?q=" + str);
xhr.send();
```

ÜBUNG: KLASSISCHE WEBSEITEN

Was sind charakteristische Merkmale des klassischen Webseiten-Modells?

- [ ] Die Benutzerinteraktion wird durch Ladezeiten unterbrochen
- [ ] Jede wesentliche Interaktion führt zu einem Seitenwechsel
- [ ] Daten werden im Hintergrund nachgeladen
- [ ] Der Server liefert immer komplette HTML-Seiten

ÜBUNG: VORTEIL VON AJAX

Was ist der Hauptvorteil von AJAX gegenüber dem klassischen Webseiten-Modell?

- [ ] Schnellere Serverantworten
- [ ] Einfachere Implementierung
- [ ] Keine vollständigen Seitenneuladungen bei Interaktionen
- [ ] Geringerer Serververbrauch

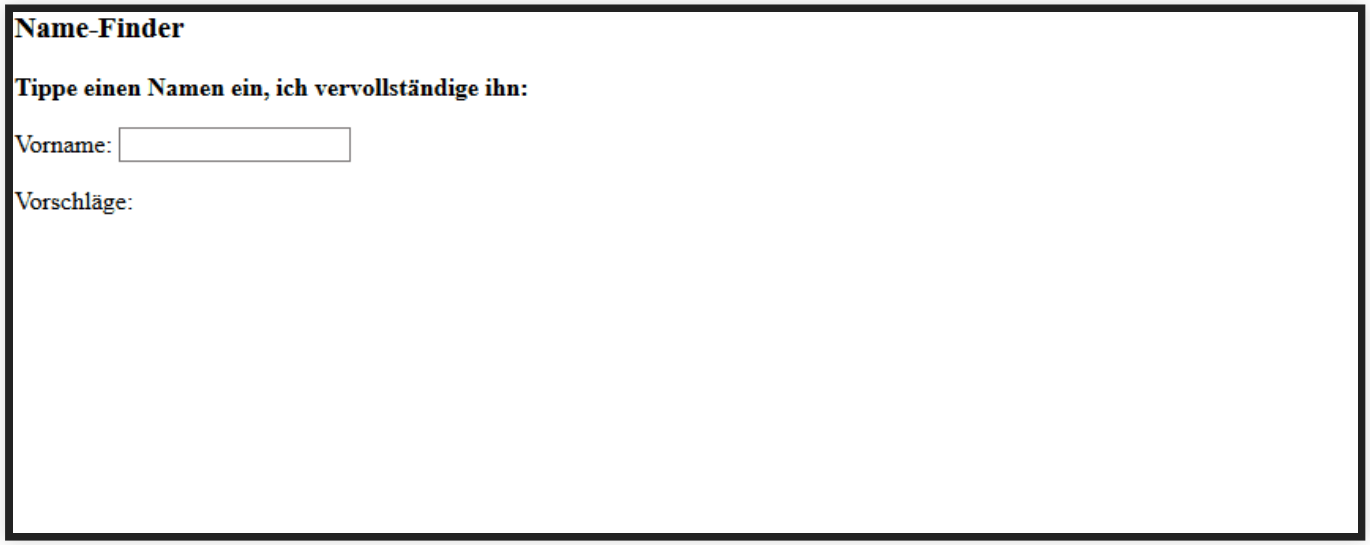
ÜBUNG: FUNKTIONSWEISE VON AJAX

Welche Aussagen stimmen?

- [ ] Das Ereignis `readystatechange` tritt mehrmals auf
- [ ] Die Webseite pausiert, bis die Antwort nach dem Aufruf von `send` verfügbar ist
- [ ] Das Ereignis `readystatechange` tritt genau einmal auf
- [ ] Die Verarbeitung der Antwort von `send` erfolgt Asynchron durch das Ereignis `readystatechange`
- [ ] Die Verwendung von `XMLHttpRequest` führt zu einer Weiterleitung

8.2 AJAX-BEISPIEL

Anhand eines kleinen Beispiels soll die Anwendung von AJAX mit einem PHP-Skript veranschaulicht werden.



Nutzen Sie das bereitgestellte Projekt-Archiv namen.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Erläuterung der Funktionsweise: Client-Teil (namen.html)

```
<h3>Name-Finder</h3>
<p><b>Tippe einen Namen ein, ich vervollständige ihn:</b></p>
<form>
    Vorname: <input type="text" onkeyup="zeigeHinweis(this.value)">
</form>
<p>Vorschläge: <span id="txtHinweis"></span></p>
```

- Wird eine Taste losgelassen ("keyup"), so wird `zeigeHinweis()` aufgerufen.
- Die Funktion `zeigeHinweis()` holt die passenden Namen vom Server und trägt diese in das span-Element mit der id "txtHinweis" ein.

```
function zeigeHinweis(praefix) {
    const span = document.getElementById("txtHint");
    if (praefix.length == 0) {
        span.innerHTML = "";
    } else {
        var xmlhttp = new XMLHttpRequest();
        xmlhttp.onreadystatechange = function () {
            if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {
                span.innerHTML = xmlhttp.responseText;
            }
        };
        xmlhttp.open("GET", "gibPraefix.php?praefix=" + praefix, true);
        xmlhttp.send();
    }
}
```

- Ist die Eingabe leer, so wird der Server-Aufruf gespart und das Hinweis-Feld wird zurückgesetzt.
- Der Server wird mit dem Präfix als Query-String aufgerufen.
- Die Antwort des Servers wird in dem "Hinweis-span" gespeichert.

Erläuterung der Funktionsweise: Server-Teil (gibPraefix.php)

```php
session_start();
if (isset($_SESSION['namen'])) {
    $namen = $_SESSION['namen'];
} else {
    if (($handle = fopen("namen.csv", "r")) !== FALSE) {
        while (($data = fgetcsv($handle, 1000, ";")) !== FALSE) {
            $namen[] = $data[1];
        fclose($handle);
    }
    $namen = array_unique($namen);  // Doubletten löschen
    sort($namen);                   // Sortieren
    $_SESSION['namen'] = $namen;
}
```

- Zu Beginn wird das eingelesene Namensarray aus der Session wieder hergestellt.
- Beim ersten Aufruf: Die Namen werden aus der Datei "namen.csv" gelesen, das resultierende Array wird unter "namen" in der Session gespeichert.

```php
$praefix = $_REQUEST["praefix"];
$hinweis = "";
if ($praefix !== "") {
    $praefix = strtolower($praefix);
    foreach($namen as $name) {
        $n = strtolower($name);
        if (strpos($n, $praefix) === 0) {
            if ($hinweis === "") {
                $hinweis = $name;
            } else {
                $hinweis .= ", $name";
            }
        }
    }
}
echo $hinweis === "" ? "kein Treffer!" : $hinweis;
```

- Das Suchpräfix wird aus dem REQUEST-Array gelesen.
- Der Hinweisstring wird in einer Iteration über alle Namen durch Anhängen gefundener Namen zusammengestellt.
- Der Hinweisstring wird zurück gesendet.

ÜBUNG: WIE FUNKTIONIERT DAS NAMEN-BEISPIEL

Welche Aussagen stimmen?

- [ ] Die Ausgabe wird aktualisiert, wenn eine Taste auf der Webseite gedrückt wurde
- [ ] Die Ausgabe wird aktualisiert, wenn eine Taste im Eingabefeld gedrückt wurde
- [ ] Der Script in `gibPraefix.php` ermöglicht über einen Parameter die Filterung einer Liste von Namen, das Ergebnis wird durch Kommas getrennt ausgegeben
- [ ] Der Script in `gibPraefix.php` erzeugt eine HTML-Liste
- [ ] Der Script in `gibPraefix.php` liefert immer alle Namen, die dann mit JavaScript gefiltert wird

ÜBUNG: WANN ENTSTEHEN AJAX-ANFRAGEN

Wann wird die AJAX-Anfrage im Beispiel ausgelöst?

- [ ] Beim Verlassen des Eingabefelds
- [ ] Bei jedem Klick auf das Eingabefeld
- [ ] Beim Absenden des Formulars
- [ ] Bei jedem Loslassen einer Taste im Eingabefeld (onkeyup)

ÜBUNG: OPTIMIERUNGEN IM BEISPIEL

Welche Optimierungen enthält das Beispiel?

- [ ] Bei leerem Eingabefeld wird keine Server-Anfrage gesendet
- [ ] Die Suche wird clientseitig durchgeführt
- [ ] Duplikate werden aus der Namensliste entfernt
- [ ] Die Namen werden serverseitig in einer Session zwischengespeichert

ÜBUNG: PRODUKTSUCHE

Ein Online-Shop möchte eine Produktsuche mit Auto-Vervollständigung implementieren. Beschreiben Sie anhand des gegebenen HTML-Codes, wie Sie diese Funktionalität mit AJAX umsetzen würden. Gehen Sie dabei auf das Event-Handling, die AJAX-Anfrage und die Verarbeitung der Serverantwort ein. Orientieren Sie sich am Namen-Beispiel.

```
<form>
    <input type="text" id="search" name="search">
    <button>Suchen</button>
    <div id="results"></div>
</form>
```

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

8.3 EINSATZ VON AJAX



Der im Beispiel gezeigte "händische" Einsatz von AJAX ist in grosseren Projekten nicht sinnvoll, weil

- typischerweise Browser-Spezifika berücksichtigt werden müssen und
- die JS-Programmierung sehr "low-level", d.h. auf niedrigem Abstraktionsniveau ist.

Es gibt daher viele JavaScript-Bibliotheken wie z.B. jQuery, die eine einfache AJAX-Nutzung ermöglichen.

Frameworks wie Angular unterstützen AJAX ebenfalls, der Entwickler hat bis auf die Behandlung der Asynchronität der Rückantwort nichts mit AJAX zu tun.

Vorteile des Einsatzes von AJAX gegenüber dem klassischen Ansatz

- Optimale User Experience möglich
- Kein Browserplugin (z.B. Flash) nötig (lediglich JS muss aktiviert sein!)
- Browserunabhängigkeit durch Browsererkennung
- Bei Bibliothek/Framework: Vermeidung von Standardtests, da getestete Funktionalität genutzt wird

Nachteile des Einsatzes von AJAX

- Verwendung des Zurück-Buttons teilweise unklar
- Risiko: Webserver-Performance schlechter durch häufiges Nachladen - hier ist gutes Design gefragt!

Anmerkung: Auch klassische Ansätze wie PHP (und andere wie ASP.NET, JSF etc.) können mit AJAX arbeiten, wodurch eine Mischform vorliegt aus Seitennachladen und AJAX-Kommunikation.

ÜBUNG: EINSATZ VON AJAX

Welche Aussagen stimmen?

- [ ] Es ist ein zusätzliches Plugin notwendig, damit AJAX in einem Browser funktioniert
- [ ] Es sollte in jedem Fall die Verwendung AJAX bevorzugt werden
- [ ] Bibliotheken wie jQuery bieten vereinfachte APIs für den Umgang mit AJAX
- [ ] Die Verwendung von AJAX und des Zurück-Buttons kann unklar sein

8.4 ÜBUNGEN ZU AJAX

ÜBUNG: WIE IST MEINE IP?

Wie ist meine öffentliche IP? Häufig sind lokale Netzwerke in speziellen IP-Bereichen organisiert - die Übersetzung in eine öffentliche IP findet letztlich durch den Router statt. Daher ist nicht selten die Frage, wie ist eigentlich meine öffentliche IP?

Nutzen Sie das bereitgestellte Projekt-Archiv ajax-ip.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.



Mit der Übung soll dieses Verhalten genauer untersucht werden unter Verwendung von AJAX. Eine kleine Webseite, die die aktuelle IP anzeigt und ein PHP-Skript, der die IP des aufrufers ausgibt. Der PHP-Skript ist vorbereitet und befindet sich in `ip.php`.



AJAX-Abfrage realisieren

- Nutzen Sie das DOM und finden Sie das HTML-Element mit der ID `ip`
- Erzeugen Sie einen JavaScript-Block und erstellen Sie eine neue `XMLHttpRequest`-Instanz
- Beschreiben Sie mit der Funktion im `onreadystatechange`-Attribut, was passiert, wenn der Aufruf erfolgreich war
- Setzen Sie den Inhalt des HTML-Element mit der ID `ip` mit dem Ergebnis des Aufrufs
- Verwenden Sie hierfür am gefundenen HTML-Element das Attribut `innerHTML` und an der `XMLHttpRequest`-Instanz das Attribut `responseText`
- Rufen Sie die Datei `ip.php` auf



Welche IP sehen Sie hier?

Das ist nicht die öffentliche IP, warum?

Änderen Sie die Adresse, von der die IP geladen wird

Versuchen Sie die Adresse (der zweite Parameter im Methodenaufruf `open`) auf `https://ipecho.net/plain` zu ändern - was passiert jetzt und welche IP ist jetzt zu sehen?

Alternative:

Öffnen Sie die Seite auf ihrem Smartphone, während ihr Webserver auf einem Rechner (Desktop, Laptop) in ihrem Home-WLAN läuft. Die Adresse wäre dann http:[Name ihres Rechners]:[Port, auf den der Webserver hört]/ajax-ip-exercise/index.html - welche IP ist jetzt zu sehen?

ÜBUNG: PROTOKOLL MIT AJAX

Die Übung für das Protokoll aus dem PHP-Kapitel wird im Folgenden so erweitert, dass eine Anwendung unter Verwendung von AJAX entsteht.

Nutzen Sie das bereitgestellte Projekt-Archiv ajax-protocol.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Analysieren Sie den aktuellen Stand - was hat sich geändert?

- Die gesamte Logik befindet sich nun in `api.php` (vorher `index.php`)
- Es wird eine andere Form des Zugriffs auf die Anfragedaten verwendet (vgl. Controller.php Zeile 34f)
- Es soll JSON produziert werden (nicht mehr HTML) und entsprechend eine Header-Information ergänzt (vgl. Controller.php Zeile 44)
- Fehlermeldungen werden in einem eigenen View behandelt (vgl. Controller Zeile 54, Zeile 6 und in ErrorView.php)
- Der View zur Bereitstellung der Liste ist aktuell Leer - und muss implementiert werden.

Alle Einträge ausgeben

- Öffnen Sie die Klasse `ProtocolListView`
- Erzeugen Sie eine Ausgabe die validen JSON-Syntax verwendet, und wie im folgenden Beispiel aussieht:

```
{"entries": [
    "Test (10:12:08 07.09.2020)",
    "Wuff (10:13:44 07.09.2020)"
]}
```

Hinweis: Sie können dies manuell mittels echo und Ausgabe in der passenden Struktur realisieren oder einen Blick auf `json_encode` werfen

Client analysieren

- Öffnen Sie die Datei `index.html`
- Vorbereitet ist das Formular und eine leere Liste mit der ID `message`
- Ebenso bereit liegen JavaScript-Funktionen:
  - zum Leeren der Liste
  - zur Erzeugung eines neuen Li-Elements für Listen
  - zur Erzeugung einer XMLHttpRequest-Instanz für JSON-basierte POST-Anfragen, Verwendung kann wie folgt aussehen:

```
createRequest("list", "", function(result) {
    // Do something with result...
});
// ... oder ...
createRequest("save", "New Message", function(result) {
    // Do something with result...
});
```



Client erweitern (1)

- Implementieren Sie eine Funktion `fillList` mit einem Attribut `entries`
- Diese soll die Funktion `clearList` aufrufen
- Die Liste mit der ID `message` aus dem DOM beziehen
- Über das Attribut `entries` iterieren und für jeden Eintrag die Funktion `createListElement` aufrufen und das Ergebnis des Aufrufs mittels der Funktion `appendChild` an die Liste anfügen

Client erweitern (2)

- Implementieren Sie eine Funktion `onLoad`
- Diese soll die Funktion `createRequest` aufrufen mit der Aktion `list`
- In der Callback-Funktion soll anschließend die Funktion `fillList` mit den Einträgen aufgerufen werden - der Parameter `result` enthält das gesamte JSON-Objekt, werfen Sie einen Blick auf das JSON-Beispiel und übergeben Sie nur das Array!
- Ergänzen Sie abschließend außerhalb der Funktion `onLoad` den folgenden Aufruf, damit diese Funktion beim Start der Webseite einmalig aufgerufen wird:

```
window.addEventListener("load", onLoad);
```



Client erweitern (3)

- Implementieren Sie eine Funktion `addProtocolEntry` (diese wird im Button-OnClick aufgerufen)
- Das Eingabefeld mit der ID `message` aus dem DOM beziehen
- Die aktuelle Eingabe über das Attribut `value` aus dem Eingabefeld beziehen
- Falls die Eingabe leer ist, mittels `alert` eine Meldung ausgeben und abbrechen
- Die Funktion `createRequest` aufrufen mit der Aktion `save` und der entsprechenden Nachricht als zweites Attribut
- In der Callback-Funktion soll anschließend wieder die Funktion `fillList` mit den Einträgen aufgerufen werden
- Für den Fall, dass die Antwort ein Attribut `error` enthält, kann diese mittels `alert` angezeigt werden



8.5 FETCH API



Die Fetch-API ist eine modernere und mächtigere Methode zur Verarbeitung von Netzwerkanfragen in JavaScript. Sie verwendet Promises und ein neues API-Design und macht den Umgang mit Netzwerkanfragen einfacher und flexibler.

Vor der Erläuterung der Fetch-API schauen wir uns an, was Promises genau sind.

PROMISES: GRUNDIDEEN

- Ein Promise ist ein Objekt zur Unterstützung für asynchrone Programmierung
- Annahme: wir wollen eine Funktion `f` aufrufen, die ihr Ergebnis asynchron liefert, d.h. `f` kehrt ohne das erwartete Ergebnis zurück, während im Hintergrund Berechnungen oder eine Netzwerkanfrage laufen.
- `f` liefert statt des noch nicht vorliegenden Ergebnisses ein Promise-Objekt, an dem man beliebig viele Handler-Funktionen und eine Fehlerbehandlungsfunktion registrieren kann

Vorteile bringen Promises insbesondere durch das Vermeiden von geschachtelten Callback-Handlern, siehe z.B. hier .



Ein Promise ...

- stellt das künftige Ergebnis einer Berechnung bzw. einer Netzwerkanfrage dar
- bietet eine Methode `then`, mit der mehrere Funktionen zur Verarbeitung des Ergebnisses registriert werden können
- bietet eine Methode `catch`, mit der eine Funktion zur Fehlerbehandlung registriert werden kann

PROMISES-BEISPIEL 1

```
const timer = (resolve, reject) => {
    setTimeout(() => {
        const n = Math.random();
        if (n > 0.5) {
            resolve("Erfolg nach " + timeout + "ms, Wert: " + n);
        } else {
            reject(new Error("Fehler nach " + timeout + "ms, Wert: " + n));
        }
    }, 2000);
};
```

- Die Funktion `timer` definiert, wie Daten abgerufen und geliefert werden: nach 2 Sekunden wird entweder eine Zufallszahl zurückgegeben oder ein Error-Objekt
- Es werden zwei Promise-Funktionen `resolve` und `reject` übergeben
- Wenn erfolgreich, wird die Funktion `resolve` aufgerufen
- Wenn nicht erfolgreich, wird die Funktion `reject` aufgerufen



Beispiel zur Verwendung

```
const promise = new Promise(timer);
promise
    .then(res => console.log(res))
    .catch(err => console.err(err));
```

Wird der Promise verwendet, kann mittels `then` und `catch` definiert werden, was passiert, wenn `resolve` oder `reject` aufgerufen wird

PROMISES-BEISPIEL 2

Nutzen Sie das bereitgestellte Projekt-Archiv promises.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Das Beispiel erzeugt 10 Promises, um verschiedene Ergebnisse in der JS-Console anzuzeigen. Die folgenden Ausführungen beziehen sich jeweils auf eine Promise-Erzeugung.

Promises-Beispiel 2: Promise-Erzeugung

```
const gibPerson = (resolve, reject) => {
    const n = Math.floor(Math.random(1)*100);
    if (n > 50) {
        const name = "Name" + n;
        const alter = n-40;
        resolve('{ "name": "' + name + '", "alter": ' + alter + ' }');
    } else {
        reject(new Error('Keine Daten vorhanden!'));
    }
};
const promise = new Promise(gibPerson);
```

- ein Promise-Objekt wird erzeugt. Die Funktion `gibPerson` wird dabei ausgeführt und liefert zufällig entweder ein Person-Objekt als JSON-String oder ein Error-Objekt
- resolve liefert ein Person-Objekt in JSON-Textformat als Ergebnis, das mit "then" verarbeitet werden kann
- reject produziert ein Error-Objekt, das mit "catch" behandelt werden kann

Promises-Beispiel 2: Verarbeitung

```
promise
.then((jsonText) => {
    return JSON.parse(jsonText);
})
.then((person) => {
    if (person.alter > 40) {
        throw new Error("Die Person ist zu alt: alter = " + person.alter);
    }
    console.log("name = " + person.name + ", alter = " + person.alter);
})
.catch((error) => {
    console.log('Fehler: ' + error.message);
});
```

- Schritt 1: JSON-Text in Person-Objekt umwandeln, der return-Wert wird im nächsten "then" verarbeitet
- Schritt 2a: Person-Objekt prüfen, ein Fehler wird in "catch" behandelt
- Schritt 2b: Person-Objekt ausgeben
- Fehlerbehandlung: Fehler ausgeben

ÜBUNG: VERWENDUNG EINES PROMISE

Welche Aussagen treffen auf Promises zu?

- [ ] Es sind mehrere catch()-Aufrufe möglich für die Fehlerbehandlung
- [ ] Sie ermöglichen die asynchrone Verarbeitung von Operationen
- [ ] Sie können mehrere then()-Handler registrieren
- [ ] Sie können nur für AJAX-Anfragen verwendet werden



ASYNCHRONE NACHRICHTENVERARBEITUNG MIT FETCH-API

Mit `fetch()` kann man Netzwerkanfragen versenden und dann mit der Promise-Syntax auf die Antwort reagieren. `fetch()` gibt immer ein Promise zurück, das entweder erfolgreich aufgelöst wird, wenn die Anfrage erfolgreich ist, oder abgelehnt wird, wenn ein Fehler auftritt.

FETCH-BEISPIEL: GET-ANFRAGE

Das Beispiel aus Abschnitt 9.1 sieht mit der Fetch-API so aus:

```
fetch("gethint.php?q=" + str)
.then(function(response) {
    if (!response.ok) {
        throw new Error('Fehler beim Laden der Daten!');
    }
    document.getElementById("txtHint").innerHTML = response.text();
})
.catch(function(error) {
    alert('Fehler beim Laden: status = ' + error.message);
});
```

- Aufruf von "gethint.php" mit Query-Parameter
- Falls kein 200er Http-Code zurück kam, wir ein Error erzeugt, der in "catch" behandelt wird
- Verarbeitung des Ergebnistextes: Anzeige in Element mit Id "txtHint"
- Fehlerbehandlung: Popup mit Fehlermeldung

FETCH-BEISPIEL: POST-ANFRAGE

Ein Beispiel für eine POST-Anfrage mit der Fetch-API:

```
fetch("save-user.php", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: "Max", alter: 25 })
})
.then(function(response) {
    if (!response.ok) {
        throw new Error('Fehler beim Senden der Daten!');
    }
    return response.text();
})
.then(function(text) {
    document.getElementById("txtResult").innerHTML = text;
})
.catch(function(error) {
    alert('Fehler beim Senden: status = ' + error.message);
});
```

```
fetch("save-user.php", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: "Max", alter: 25 })
})
.then(function(response) { ... })
.then(function(text) { ... })
.catch(function(error) { ... });
```

Aufruf von `save.php` mit POST-Methode und JSON-Daten im Body (als String, erzeugt aus `JSON.stringify({ name: "Max", alter: 25 })`).

```
fetch("save-user.php", { ... })
.then(function(response) {
    if (!response.ok) {
        throw new Error('Fehler beim Senden der Daten!');
    }
    return response.text();
})
.then(function(text) {
    document.getElementById("txtResult").innerHTML = text;
})
.catch(function(error) {
    alert('Fehler beim Senden: status = ' + error.message);
});
```

- Falls kein 200er Http-Code zurück kam, wird ein Error erzeugt, der in `catch` behandelt wird
- Verarbeitung des Ergebnistextes: Anzeige in Element mit Id `txtResult`
- Fehlerbehandlung: Popup mit Fehlermeldung

ÜBUNG: FETCH-API VS. XMLHTTPREQUEST

Was ist der Vorteil der Fetch-API gegenüber XMLHttpRequest?

- [ ] Sie benötigt weniger Serverressourcen
- [ ] Sie bietet eine modernere Promise-basierte Syntax
- [ ] Sie ist schneller
- [ ] Sie funktioniert in mehr Browsern

ÜBUNG: XMLHTTPREQUEST ZU FETCH

Wandeln Sie den folgenden XMLHttpRequest-Code in eine äquivalente Version mit der Fetch-API um:

```
let xhr = new XMLHttpRequest();
xhr.onload = function () {
    if (xhr.status == 200 && xhr.readyStatus == 4) {
        console.log("Antwort: " + xhr.responseText);
    } else {
        console.log('Fehler: ' + xhr.status);
    }
};
xhr.open("GET", "data.php?id=123");
xhr.send();
```

Wie sieht der gegebene XMLHttpRequest-Code mit der Fetch-API aus?

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

ASYNC UND AWAIT

Was ist `async`/`await`?

```
async function doSomething() {
    let result = await new Promise(...);
}
```

- Syntaktischer Zucker über Promises, macht asynchronen Code lesbarer wie synchronen Code
- `async`-Funktionen: Geben immer ein Promise zurück
- `await`-Operator: Pausiert die Ausführung bis Promise aufgelöst ist, kann nur in async-Funktionen verwendet werden

Einfaches Beispiel: Umschreiben eines der vorherigen Fetch-Beispiele

```
function ladeHint(str) {
    fetch("gethint.php?q=" + str)
    .then(function(response) { ... })
    .catch(function(error) { ... });
}
```

```
async function ladeHint(str) {
    const response = await fetch("gethint.php?q=" + str);
    if (!response.ok) {
        throw new Error('Fehler beim Laden der Daten!');
    }
    const text = await response.text();
    document.getElementById("txtHint").innerHTML = text;
}
```

Wird die Funktion mit `async` definiert und kann der Fetch-Aufruf (der ein Promise liefert) mittels `await` verwendet werden. Als Rückgabe erhält man das, was im `resolve` übergeben wird, bzw. im `then` verarbeitet werden würde.

Fehlerbehandlung

```
async function speicherePerson() {
    try {
        const response = await fetch("save-user.php", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ name: "Max", alter: 25 })
        });
        if (!response.ok) throw new Error('Fehler beim Senden!');
        const text = await response.text();
        document.getElementById("txtResult").innerHTML = text;
    } catch (error) {
        alert('Fehler: ' + error.message);
    }
}
```

Wird ein Fehler mittels `reject` oder `throw new Error` erzeugt, kann diese bei await mittels try-catch behandelt werden.



8.6 ÜBUNGEN ZU PROMISES UND ZUR FETCH-API

ÜBUNG: PROMISES

In der folgenden Übung soll nach einer zufälligen Wartezeit eine Zufallszahl erzeugt werden. Nur die geraden Zahlen sollen auf die Console ausgegeben werden.

Nutzen Sie das bereitgestellte Projekt-Archiv promises-numbers.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Gegeben sei folgender Lambda-Ausdruck (siehe Download):

```
// Liefert nach einer zufälligen Zeit einen
// JSON-String der Art: "{ number: 1234 }"
const zahlenLieferant = (resolve, reject) => {
    const number = Math.trunc(Math.random() * 10000);
    setInterval(function() {
        resolve(`{ "number": ${number} }`);
    }, number);
}
```

- Bei Ausführung wird eine Zufallszahl N von 0 bis 9999 erzeugt
- nach N/1000 Sekunden wird ein JS-Objekt mit der Zahl erzeugt und zurückgegeben

Aufgabe: Erzeugen Sie 10 Promises mit obigem Lambda und geben Sie die geraden Zahlen aus

